



Gymnázium J. K. Tyla

DarkFactory: Artificial Intelligence in Practice – Agentic and Harness Engineering

THESIS

Author: Patrik Marius, 4.D
Supervisor: Michal Dočekal

Extent: 1011 words / 6911 characters

Declaration

I declare that I prepared this specialized thesis independently under the supervision of the supervisor named on the title page. All sources used are listed in the bibliography and information derived from them is cited in the text. I agree that the printed version may be archived at Gymnázium J. K. Tyla and used there as a reference for future student work or for presenting education at GJKT.

In Hradci Králové on

.....

Author signature:

Annotation

[EN] This thesis examines the principles of agentic engineering: effective engineering practices for development with artificial intelligence through agentic systems and the architecture of these systems. The practical contribution of the thesis is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize automation of the development lifecycle, from interpreting requirements in GitHub Issues, through planning, to code development and merging changes. The thesis reflects that current agentic systems cannot be regarded as fully autonomous: language models require deterministic guardrails, context management, and human involvement (*Human-in-the-loop*).

Keywords

Agent, Agentic Engineering, Agent Harness, Chatbot, Skills, Git, GitHub, Large Language Model (LLM) , ReAct Loop (Agent Loop) , Embedding

Contents

Declaration	2
Annotation	3
Keywords	4
1 Úvod	8
1.1 Motivace a vymezení problému	8
1.2 Cíl práce a výzkumné otázky	8
1.2.1 Hlavní cíl	8
1.2.2 Dílčí cíle	8
1.2.3 Výzkumné otázky	8
1.3 Metodika práce	8
2 Teoretická část: Vymezení konceptu	9
2.1 Vývojové prostředí a praxe	9
2.1.1 Správa verzí [Version Control]	9
2.1.2 Git a GitHub	9
2.1.3 Větvě (Branches)	9
2.1.4 Pull Request	9
2.1.5 Slučování změn (Commit and Merge)	9
2.1.6 Plánování [Planning]	9
2.1.7 Kontinuální integrace (CI a GitHub Actions)	9
2.1.8 Požadované kontroly (Required Checks)	9
2.2 Large Language Model (LLM) , chatboti a agenti	9
2.2.1 Úvod	9
2.2.2 Agent vs. Chatbot	10
2.2.3 Tokeny, tokenizace a Vektorová reprezentace [Embedding]	10
2.3 Agentic Engineering a Agent Harness	11
2.3.1 Úvod	11
2.3.2 Vyvolávání nástrojů [Tool Calling]	12
2.3.3 Skills	12
2.3.4 Škálování: Multiagentní systémy (Subagenti) a grafy (DAG workflows) [Scaling: Multiagent Systems (Subagents) and DAG Workflows (Graphs)]	12
3 DarkFactory - Praktická část: Architektura harnessu	13
3.1 Git a GitHub	13
4 Výsledky a diskuse	14
5 Závěr	14

Index	14
List of appendices	20
References	21

List of figures and tables

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	12
-----------	--	----



CHAPTER 1

Úvod

1.1 Motivace a vymezení problému

Ústřední inženýrská otázka této práce proto nespočívá v tom, zda jazykový model dokáže napsat fragment kódu. Zkoumáme, jaká kontrolní a dozorčí architektura — značovaná jako *Agent Harness* — musí model obklopovat, aby bylo možné jeho výstupům v produkčním repozitáři spolehlivě důvěřovat a dosáhnout vysoké míry autonomie se zachováním lidského dohledu.

1.2 Cíl práce a výzkumné otázky

1.2.1 Hlavní cíl

Vymezit teoretické principy agentického inženýrství (*agentic engineering*) a navrhnout modulární architekturu agent harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

1.2.2 Dílčí cíle

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu agent harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.2.3 Výzkumné otázky

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Vzhledem k dynamickému vývoji v oblasti autonomního softwarového vývoje práce důsledně zachovává a integruje zavedené anglické odborné názvy (např. *harness*, *pull request*, *agent loop*, *prompt engineering*, *skills* či *context rot*). Použití této terminologie je integrální součástí práce, neboť tyto anglické pojmy představují de facto celosvětové průmyslové standardy (*industry standards*), jejichž doslovný český překlad by byl nejednoznačný, zavádějící či v rozporu s běžnou inženýrskou praxí.

Postup práce sleduje strukturu inženýrského cyklu:

- 1. Analýza konceptu: Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- 2. Návrh architektury: Formulace modulárního modelu agent harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- 3. Kritické zhodnocení: Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.



CHAPTER 2

Teoretická část: Vymezení konceptu

2.1 Vývojové prostředí a praxe

2.1.1 Správa verzí [Version Control]

2.1.2 Git a GitHub

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém **Git** v kombinaci s platformou **GitHub**.

Klíčové komponenty infrastruktury zahrnují:

- Distribuovaný systém Git (1): Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- Platforma GitHub: Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - Zadávání a sledování úkolů (Issues): Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - Revize změn (Pull Requests): Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - Automatizace (GitHub Actions): Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

Agent v tomto pojetí nevystupuje jako černá skříňka s proprietárním protokolem, nýbrž jako standardní přispěvatel, který plně respektuje běžné vývojářské zvyklosti a nástroje.

2.1.3 Větve (Branches)

2.1.4 Pull Request

2.1.5 Slučování změn (Commit and Merge)

2.1.6 Plánování [Planning]

2.1.7 Kontinuální integrace (CI a GitHub Actions)

2.1.8 Požadované kontroly (Required Checks)

2.2 Large Language Model (LLM), chatboti a agenti

2.2.1 Úvod

V agentickém softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodérový

transformer (*Decoder-only*), jehož typickými představiteli jsou moderní modely řad Claude, GPT či DeepSeek (2). Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- Autoregresivní predikce: Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (tokeny).
- Stochastická povaha: Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném agent harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť.

2.2.2 Agent vs. Chatbot

Chatbot★ — [cz] Systém založený na jazykovém modelu určený primárně k textové interakci s uživatelem; odpovídá na jednotlivé požadavky, ale sám o sobě nedisponuje autonomní prováděcí smyčkou ani nástroji pro samostatnou modifikaci okolního prostředí.. **Agent**★ — [cz] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí víceukrokové akce směřující k dosažení zadaného inženýrského cíle.. Rozdíl mezi nimi nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí.

Srovnání obou přístupů:

- Konverzační chatbot:
 - Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
 - Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
 - Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.
- Autonomní agent:
 - Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repozitářem.
 - Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
 - Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

2.2.3 Tokeny, tokenizace a Vektorová reprezentace [Embedding]

Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- Tokeny a tokenizér: Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér (nejčastěji na bázi algoritmu Byte Pair Encoding, BPE).

- **Embedding**★ — [cz] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné sémantické vztahy mezi reprezentacemi. (např. vektorová analogie král — muž + žena $\approx$ královna).
- Jazyková asymetrie tokenizace: Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

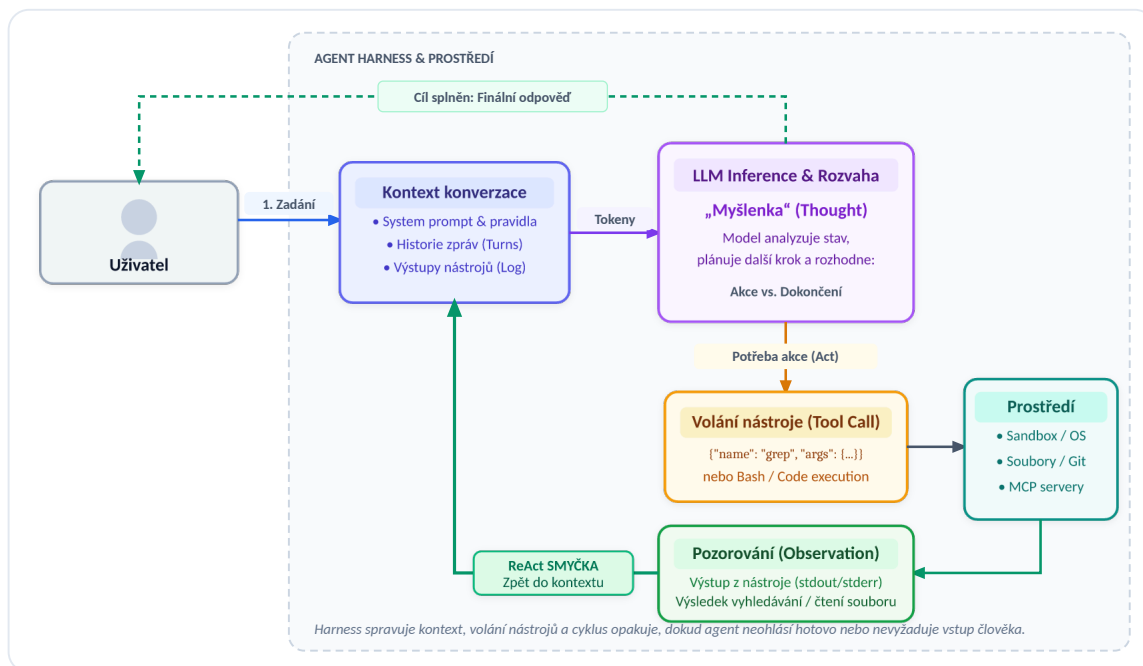
Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

2.3 Agentic Engineering a Agent Harness

2.3.1 Úvod

V terminologii agentického inženýrství používá tato práce pojem **Agent Harness**★. [cz] Agentní harness — aplikační a orchestrační vrstva obklopující inferenční jádro modelu, která zajišťuje běhové prostředí nástrojů, dynamickou správu kontextového okna, bezpečnostní mantinely, práci se stavem a deterministické řízení životního cyklu požadavku.. Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; veškerou orchestraci, práci se soubory a řízení bezpečnosti zajišťuje harness.

Ústřední komponentou a hlavní prováděcí funkcí, která v architektuře harnessu řídí samotný běh a iterativní koordinaci agenta v reálném vývojovém prostředí, je **ReAct Loop (Agent Loop)** ★. [cz] Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí..



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

2.3.2 Vyvolávání nástrojů [Tool Calling]

2.3.3 Skills

Se vzrůstající komplexitou úloh nelze veškeré instrukce, skripty a doménové znalosti vkládat do základního systémového promptu. K modulárnímu rozšíření schopností agenta slouží **Skills** — [CZ] Znovupoužitelné modulární balíčky instrukcí (typicky definovaných v souboru SKILL.md), procedurálních pravidel a volitelných pomocných skriptů či zdrojů, které harness dynamicky načítá do kontextu agenta podle povahy řešeného úkolu..

2.3.4 Škálování: Multiagentní systémy (Subagenti) a grafy (DAG workflows) [Scaling: Multiagent Systems (Subagents) and DAG Workflows (Graphs)]

Monolitická agentní smyčka selhává při řešení komplexních, vícefázových úloh. Pro spolehlivé škálování se v moderních systémech uplatňuje hierarchická dělba práce a formalizace procesu do podoby grafu.



CHAPTER 3

DarkFactory - Praktická část:

Architektura harnessu

Úvod

3.1 Git a GitHub



CHAPTER 4

Výsledky a diskuse



CHAPTER 5

Závěr

Index

Agent
Agentic Engineering
Agent Harness
Chatbot
Context Rot
Skills
Retrieval-Augmented Generation (RAG)
Git
GitHub
GitHub Actions (Actions)
Workflow-graph Engineering (Graph Engineering)
Execution-loop Engineering (Loop Engineering)
Large Language Model (LLM)
Context Compaction (Compaction)
Context Engineering
Context Window
Key-Value Cache (KV Cache)
Model Context Protocol (MCP)
Directed Acyclic Graph (DAG)
Pull Request
Prompt Engineering
Continuous Integration (CI)
Plugins
Script
Commit Squashing (Squash)
Branch Merge (Merge)
ReAct Loop (Agent Loop)
Software Engineering
Software Container (Container)
Version control
Interaction Turn (Turn)
Token
Tokenizer
Transformer Architecture (Transformer)
Event Hook (Hook)

[Embedding](#)

[Repository Branch](#) ([Branch](#))

[Human-in-the-loop](#) ([HITL](#))

A

Agent

[EN] A software system driven by a language model and equipped with tools that independently plans, observes its environment, and performs multi-step actions toward a specified engineering goal.

Agentic Engineering

[EN] An engineering discipline focused on designing, orchestrating, and operating agentic systems around language models, including tools, context, execution loops, guardrails, and human oversight.

Agent Harness

[EN] Agent harness — an application and orchestration layer surrounding a model's inference core that provides the tool runtime, dynamic context-window management, guardrails, state handling, and deterministic control over the request lifecycle.

C

Chatbot

[EN] A language-model-based system designed primarily for text interaction with a user; it responds to individual requests but does not by itself provide an autonomous execution loop or tools for independently modifying the surrounding environment.

D

Context Rot

[EN] Degradation in a model's attention and reasoning quality caused by long, noisy, or internally competing context, leading to missed instructions and loss of relationships between facts.

Skills

[EN] Reusable modular packages of instructions (typically defined in a SKILL.md file), procedural rules, and optional helper scripts or resources that a harness dynamically loads into an agent's context for a particular class of task.

G

Retrieval-Augmented Generation (RAG)

[EN] An architecture in which a system retrieves relevant information from an external source before or during generation and places it into model context so the output can be grounded in the retrieved data.

Git

[EN] A distributed version-control system that records project history, supports branching and merging, and enables deterministic return to earlier repository states.

GitHub

[EN] A platform for hosting Git repositories and coordinating the software-development lifecycle through features such as Issues, Pull Requests, and CI/CD automation.

GitHub Actions (Actions)

[EN] GitHub's automation platform for running declared workflows and their jobs in response to repository events or manual dispatch.

I

Workflow-graph Engineering (Graph Engineering)

[EN] The design of agentic or automation workflows as explicit graphs of nodes, dependencies, and transitions rather than as one unconstrained loop.

Execution-loop Engineering (Loop Engineering)

[EN] The design and control of an agent's iterative execution loop, including state transitions, termination conditions, budgets, retries, escalation, and the connection between model decisions and tools.

J

Large Language Model (LLM)

[EN] A large language model is a neural model trained on large-scale textual data that autoregressively processes and generates token sequences. In this thesis it serves as the inference-based cognitive core of an agentic system.

K

Context Compaction (Compaction)

[EN] The process of reducing active context, typically by summarizing, selecting, or replacing older history with a more compact representation so execution remains within the context window.

Context Engineering

[EN] The systematic design, selection, ordering, and lifecycle management of information made available to a model in active context, including instructions, memory, tool results, and externally retrieved data.

Context Window

[EN] The maximum token-sequence span a model can include in active context during one inference run. In practice it limits the combined instructions, history, tool outputs, and other data supplied to the model.

M

Key-Value Cache (KV Cache)

[EN] A cache of previously computed key and value vectors in transformer attention layers that reduces repeated computation of earlier tokens during autoregressive generation.

Model Context Protocol (MCP)

[EN] Model Context Protocol — an open standard originally introduced by Anthropic for standardized communication between AI applications and external tools, resources, and data through JSON-RPC messages.

O

Directed Acyclic Graph (DAG)

[EN] A directed graph containing no directed cycle. In workflows it can explicitly represent dependencies among steps and the ordering implied by those dependencies.

P

Pull Request

[EN] A formal proposal to integrate changes from one repository branch into another, providing a place for automated checks, human review, and discussion of the proposed changes.

Prompt Engineering

[EN] An engineering discipline for systematically designing, structuring, and optimizing instructions and system prompts to guide and constrain language-model behavior.

Continuous Integration (CI)

[EN] A development practice in which changes are integrated frequently and automatically verified by builds, tests, and other checks so integration failures are detected early.

R

Plugins

[EN] Programmatic extension modules running directly in the harness environment that extend its execution core with specialized system adapters, tool drivers, and deterministic hooks.

S

Script

[EN] A file or sequence of commands intended for automated execution by an interpreter, shell, or another runtime.

Commit Squashing (Squash)

[EN] An operation that replaces multiple consecutive commits with one aggregate commit, commonly to simplify history before integrating changes.

Branch Merge (Merge)

[EN] A version-control operation that combines changes or history from two lines of development into a common resulting state; conflicts require explicit resolution.

ReAct Loop (Agent Loop)

[EN] An iterative execution cycle of an autonomous agent based on the ReAct pattern (Reasoning + Acting), in which the model alternates between reasoning, tool calls, and evaluation of observations from the runtime environment.

Software Engineering

[EN] The systematic application of engineering principles to the specification, design, implementation, verification, operation, and maintenance of software systems.

Software Container (Container)

[EN] An isolated user-space runtime packaging an application and its dependencies while sharing the host operating-system kernel; its security isolation depends on the implementation and configuration.

Version control

[EN] The management and tracking of changes to source files and other versioned artifacts so changes can be safely branched, merged, audited, and reverted when necessary.

T

Interaction Turn (Turn)

[EN] One discrete unit of interaction in a conversational or agentic protocol, such as a user message, model response, or separately recorded tool result.

Token

[EN] A discrete unit processed by a language model. A token corresponds to an entry in the tokenizer vocabulary and is represented by a numeric identifier; it need not correspond to a whole word.

Tokenizer

[EN] A component that maps text or another input into a sequence of tokens and token identifiers and, where supported, performs the reverse decoding transformation.

Transformer Architecture (Transformer)

[EN] A neural-network architecture based on attention mechanisms that models relationships among sequence elements and underlies most contemporary large language models.

U**Event Hook (Hook)**

[EN] A defined lifecycle or event point to which custom deterministic logic can be attached before, after, or in place of standard system behavior.

V**Embedding**

[EN] A multidimensional vector representation of tokens or other data in which numerical relationships between vectors capture useful semantic relationships between representations.

Repository Branch (Branch)

[EN] A named line of development in version control that allows changes to proceed separately from another history line and later be compared or merged.

Z**Human-in-the-loop (HITL)**

[EN] A design pattern in which a human operator remains part of the system's decision process through approval gates (Human Gates), especially before consequential or irreversible system operations.

List of appendices

References

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
3. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
4. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
5. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
6. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
7. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
8. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
9. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
10. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
11. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>